# A Minimal Self-Referential Graph Formalism for Internal Semantic and Temporal Structure

Shahzada Saeed

MSc Computer Science, University of London

Source code and development history: public project repository.

## Abstract

This dissertation presents a combined theoretical and practical investigation into whether a discrete computational system can internalise its own interpretation. It develops a self-referential graph formalism in which rewrite rules are encoded within the graph they transform, and examines the conditions under which useful structure persists through change. Rule-version ranks bound particular chains of self-modification; restricted program families admit termination, recovery and stable-output guarantees. Dependencies between rewrite events provide a partial temporal order, while a graph-resident lambda evaluator establishes a correspondence with a recognised model of computation. A Rust developer platform makes these results executable through program authoring, graph inspection, verified histories and hosted projects. Evaluation combines mathematical arguments, selected computer-checked proofs, independent tests and measured applications. The findings establish controlled operational reflection and conditions for stability and useful interpretation.

# Contents


1. Introduction 4
   1.1 Problem and motivation 4
   1.2 Research questions and objectives 4
2. Related work and methodology 5
   2.1 Reflection and executable descriptions 5
   2.2 Graph transformation and expressiveness 5
   2.3 Progress, observation and causality 5
   2.4 Research method 6
3. A reduced self-referential graph formalism 6
   3.1 Carrier and vocabulary 6
   3.2 Rules, quotation and packages 7
   3.3 Binding and admission 8
   3.4 Revisions, ranks and non-regress 9
   3.5 The meaning of reduction 11
4. Semantic stability and convergence 13
   4.1 Observations and fixed points 13
   4.2 A sufficient normalisation class 13
   4.3 Stability without collapse 13
   4.4 Recovery with changing policies and bodies 13
   4.5 Structural contracts and persistent work 14
   4.6 Local numerical accuracy during continuing evolution 16
5. Event dependencies and temporal structure 16
   5.1 Resource order 16
   5.2 Scope of the resource order 16
   5.3 Tracing selected influences 17
   5.4 Checked histories 17
6. Expressiveness 18
   6.1 Source and graph machine 18
   6.2 Invariant and correspondence 19
   6.3 Ranked term graphs 20
   6.4 Version-aware interpretation 21
7. Software design and implementation 21
   7.1 Architecture and semantic authority 21
   7.2 Execution and scaling 22
   7.3 Authoring and inspection 22
   7.4 Developer platform 23
   7.5 Development process 23
8. Evaluation 24
   8.1 Conformance with the formal account 25
   8.2 Performance and useful interpretation 25
   8.3 Acquired representations and renewal 27
   8.4 Threats to validity 28
9. Discussion and conclusion 28
   9.1 Research outcomes and significance 28
   9.2 Practical significance 29
   9.3 Further research and conclusion 29
Bibliography 30

# List of figures

Figure 1. An addressable relation becomes a record with two source-owned fields. 7

Figure 2. Ground replacement precedes transactional validation. Rejection preserves the original carrier. 9

Figure 3. Supervised restoration permits continuing modification. Each arrow is an edit; restoration replaces the revision produced by the preceding lower-ranked rule. 11

Figure 4. Two program families: payload-preserving oscillation (upper) and guarded normalisation (lower). The lower routes show alternatives before and after a permitted rule revision. 15

Figure 5. Events (rectangles) transmit selected ancestry through produced and read atoms (ellipses). The lower example shows a visibility influence absent from that resource path. 18

Figure 6. Readback relates graph-machine configurations to source expressions. Solid arrows are transitions; dashed arrows are readback. Push and lookup preserve the expression, while Beta reduces it. The final machine arrow abbreviates two lookups. 20

Figure 7. UML deployment view of the platform. Browser and hosted execution share the Rust engine. Components exchange typed messages, authenticated requests and stored artefacts across the indicated boundaries. 22

Figure 8. Practical graph views before and after one rewrite of the payload-preserving example. The thicker green relation is the new done record; the value record and endpoints persist. 23

Figure 9. Development timeline, 1 June–11 October 2026. Sprint 1 comprises literature review and formal foundations; Sprint 6 comprises platform development and research/integration. 24

Figure 10. Complete processing time and peak memory (a–b): medians and ranges of three measured repetitions per completed cell. The initial repeat's four incomplete 100,000-item attempts have no completion estimate. Public-data transformation (c): three fixed-policy runs and one approved-policy control. Sources: E9–E10. 27

# List of tables

Table 1. Record field schemas 7

Table 2. Representation costs 12

Table 3. Closure-machine transitions 19

Table 4. Experimental evidence and reproduction 24

# 1. Introduction

## 1.1 Problem and motivation

Standard models of computation separate syntax from semantics. Operational and denotational semantics rely on an externally specified account to confer meaning on otherwise inert symbols, while knowledge-graph technologies commonly presuppose predefined ontologies and labels. This separation raises a foundational question: how far can a discrete computational system internalise its own interpretation, so that stable behaviour and temporal ordering arise from its structural dynamics?

Graph rewriting offers a suitable setting because it treats computation as the transformation of relationships. When rules are represented within the graph, the same mechanism can change both data and the procedures governing it. Consider a workflow whose processing rule is revised while requests are pending. Understanding its behaviour requires knowing which rule version applies, whether work can still finish and which earlier changes enabled a result.

“Internal” refers to resident rule content, activation and modification; matching and transition retain an externally specified semantics. Stability is assessed against declared observations, allowing the investigation to distinguish a useful persistent result from an empty or incorrect one.

## 1.2 Research questions and objectives

The thesis is that a small graph formalism can bring executable self-description, stable observations and dependency-based temporal order into one computational account. Five questions structure the investigation, linking its foundational ambition to properties that can be proved or tested:

- **RQ1:** What is a minimal node-and-edge formalism in which a graph can encode its own rewrite rules while avoiding unbounded meta-regress?
- **RQ2:** Under what structural and operational conditions does internal rewriting terminate, converge, or reach a stable equivalence class?
- **RQ3:** Which fixed-point, normal-form and observational criteria distinguish semantic stability, persistent oscillation or divergence, and trivial collapse?
- **RQ4:** How can dependency between rewrite events induce a causal partial order, and how does that order relate to event structures and established models of concurrent computation?
- **RQ5:** Where does the formalism sit, in expressive power, relative to the untyped lambda calculus and established graph rewriting systems?

These questions guide the formal account in Sections 3–6 and the platform, evaluation and conclusions in Sections 7–9. The engineering aim is a usable environment for authoring, execution and investigation; numerical and acquired-representation studies examine useful stability in applications.

# 2. Related work and methodology

## 2.1 Reflection and executable descriptions

Classical semantics places the defining machinery outside the system being defined. Reynolds (1972) shows how interpreters inherit choices from their metalanguage, making the underlying execution mechanism an essential part of any account of internal interpretation. Smith's (1984) reflective towers expose levels of interpretation; Amin and Rompf (2018) show how staging can reduce their execution overhead. Typed quotation in reFLect connects reflection with verification (Grundy, Melham and O'Leary, 2006).

Cirstea and Kirchner (2001) make rules first-class abstractions; Clavel and Meseguer (2002) provide universal metarepresentation in conditional rewriting logic. Machado, Ribeiro and Heckel (2015) transform graph-rewriting rules through a fixed first-order/second-order architecture, while Tomita, Murata and Kurokawa (2007) propose graph metanodes containing computational rules. Reflective abstract state machines characterise broad classes of behaviour with updatable algorithm descriptions (Schewe and Ferrarotti, 2022; 2026). The present investigation builds on these ideas by bringing representation, revision-specific guarantees and execution evidence into one restricted ordinary-graph model.

## 2.2 Graph transformation and expressiveness

Double-pushout (DPO) rewriting characterises deletion and creation around a preserved interface (Ehrig, Prange and Ehrig, 2006). This supplies the basis for immutable records and explicit checks against broken references. The correspondence developed here applies after variables have been bound, when the participating graph elements have concrete types.

Term graphs represent sharing and recursion directly (Barendregt et al., 1987), with convergence studied under appropriate structural and metric assumptions (Bahr, 2012). Krivine's (2007) closure machine supplies the evaluation strategy; graphical implementations include Lippi (2007).

GP 2 provides graph programming (Plump, 2012), PORGY interactive derivations (Fernández, Kirchner and Pinaud, 2019), and Quantomatic diagrammatic reasoning (Kissinger and Zamdzhiev, 2015). This platform's evaluated distinction is the inspection of resident mutable rules under the stated admission discipline, without a claimed comparative usability advantage.

## 2.3 Progress, observation and causality

Rewriting theory supplies operational criteria for convergence: termination and local confluence imply unique normal forms (Newman, 1942). Plump's (1998) undecidability result motivates sufficient conditions for selected classes. Dijkstra's (1974) self-stabilisation extends the question to recovery from specified disturbances, while asynchronous and componentwise approaches address accuracy during continued evolution (Bertsekas, 1983; Kawan, 2025). These distinctions guide the separate progress arguments in Section 4.

Fixed-point theory requires particular care. Knaster–Tarski supplies fixed points for monotone operators on complete lattices (Tarski, 1955); its application here characterises reachable sets. Kleene's (1952) recursion theorem establishes computable self-reference. Neither result alone

establishes runtime convergence. Tarski's (1956) truth-undefinability theorem concerns an internal truth predicate, which this language does not define.

Lamport's (1978) dependency order differs from chronology. Event structures additionally describe conflict and alternatives (Nielsen, Plotkin and Winskel, 1981). A single resource history therefore gives a partial explanation, motivating the intervention analysis in Section 5. Likewise, stable symbols need not have correct interpretations: Harnad's (1990) grounding problem motivates external task oracles rather than inferring meaning from normality.

### 2.4 Research method

The investigation combined constructive theory with experimental software development. Proposed guarantees were challenged with small counterexamples before extension. A supervisor that repeatedly restores a rule's rank, for example, forced a distinction between finite revision chains and continuing rule modification. Proofs then stated the surviving assumptions; executable examples checked their consequences. Independent evaluators, finite enumeration and generated property tests supplemented these arguments (Claessen and Hughes, 2000). Performance studies measured complete processing and memory, retaining failed attempts. The literature review followed primary publications and backward citations. Evidence records E1–E13 in Section 8 link the reported experiments to their inputs, results and verification procedures.

## 3. A reduced self-referential graph formalism

The central design decision is to make a rule inspectable in the same way as the data it changes. Consider a pending request: one part of the graph describes the request, while another describes the rule that marks it done. A further rule may revise that processing rule. All three belong to one graph, called the carrier. The following definitions distinguish these roles and protect their references.

### 3.1 Carrier and vocabulary

A configuration is a finite typed attributed directed multigraph $G = (A, F, s, t, kind, role, attr)$. $A$ contains atom identities; $F$ contains source-owned fields, with $s, t : F \to A$ giving their endpoints and distinct slots 0 or 1. Preserved atoms retain their attributes and fields, so changing a record requires a fresh replacement. Identities are never reused.

The five kinds are Constructor, PatternVariable, RuleHead, Nat and Record. Constructors carry equality-only labels, variables specify permitted kinds or record roles, and rule heads identify descriptions. These three kinds are fieldless. Nat represents zero or a successor pointing to another Nat. Records connect these elements through the following schemas.

Table 1. Record field schemas

| Role | Slot 0 | Slot 1 |
|---|---|---|
| ObjectLink | Constructor | Constructor |
| Lhs, Rhs | RuleHead | Any atom |
| Rank | RuleHead | Nat |
| Active, ModeObject, ModeMeta, Revision | RuleHead | Absent |

ObjectLink records carry equality-only labels. Pattern variables respect the field schemas; rank chains are finite, acyclic and private to their rule heads.

For example, pending($a$, $b$) targets a and b; replacing it with done preserves those endpoints (Figure 1). Distinct records preserve parallel links.

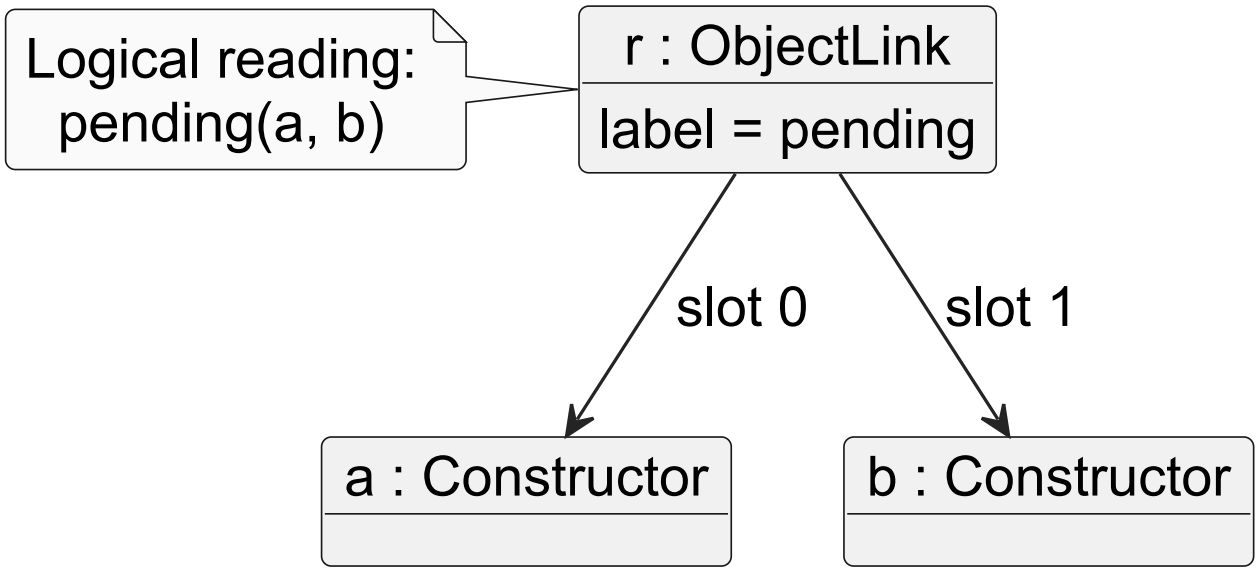



Figure 1. An addressable relation becomes a record with two source-owned fields.

## 3.2 Rules, quotation and packages

A rule describes what to match, preserve and create. Its description must remain distinguishable from the data it matches. Structural quotation provides this distinction: Lhs/Rhs membership records identify syntax belonging to a rule. A membership is operative when it is not itself quoted; every quoted membership must have an operative enclosing membership. Rootless quotation cycles are invalid, and quoted structure has one live owner.

An unquoted RuleHead is live. Each live head has a rank, mode and executable revision, with an optional Active record. Its left and right item sets $L$ and $R$ share the preserved interface $K = L \cap R$. Active sides are closed under concrete outgoing fields, and replacement variables must already occur in $L$. Object-mode rules operate on ordinary data outside all live rule representations; meta-mode rules can inspect the full carrier, including inactive descriptions.

New heads are introduced as closed fresh packages containing their metadata and syntax. Copying them without the enclosing quotation makes them live. Rep($h$) denotes the head's complete representation, including owned records, rank chain and syntax closure; BodyRep($h$) excludes its

current executable Revision. A meta-rule can change another rule's mode, thereby extending what it can inspect. Application access permissions are enforced separately by the platform.

## 3.3 Binding and admission

An active source $p$ selects an injective assignment $\sigma$ for its left-hand pattern. Concrete templates match types, attributes and incidence; variables match permitted sorts. Distinct pattern identities receive distinct images. Variables may bind freely within their sorts or, with AppliedRule binding, refer specifically to the executing meta-rule.

Let $C = \sigma(L \backslash K)$ be the consumed atoms. Following outgoing fields completes the matched context $L_\sigma = \mathrm{cl}_G(\sigma(L))$; the preserved context is $K_\sigma = L_\sigma \backslash C$. Object-mode visibility covers this completed context. The replacement is $R_\sigma = K_\sigma \cup \mathrm{fresh}(R \backslash K)$, with variables substituted and fresh references resolved together.

No surviving field may target a consumed atom. Collect admission as a small-step judgment (Plotkin, 2004):

$$\frac{\mathrm{Valid}(G) \quad \mathrm{Active}(p) \quad \mathrm{Bind}(\sigma) \quad \mathrm{GroundVisible}(\sigma) \quad \mathrm{Dangling} \quad \mathrm{FreshStep}(G, G') \quad \mathrm{Valid}(G') \quad \mathrm{Lifecycle} \quad \mathrm{GC}}{G \xrightarrow{e} G'}$$

The event $e$ records the applied revision and affected resources (§5.1). Only a checked candidate becomes visible (Figure 2).

**Lemma 3.1 (shared-identity interface).** A finite ground span with injective type-preserving legs admits literal side intersection. Rename non-interface parts apart and identify corresponding interface elements; transporting incidence and attributes gives an isomorphic span.

**Proposition 3.2 (restricted DPO correspondence).** Expand the grounded sets with their source-owned fields. For injective ground matching and immutable attributes, the delete–preserve–create construction is the DPO result exactly when the dangling condition holds.

*Proof.* Injectivity removes identification obstructions. Deleting consumed sources removes their fields; any remaining reference to a deleted atom is exactly a dangling obstruction. Its absence supplies the pushout complement, and adjoining fresh elements supplies the second pushout.

This correspondence concerns grounded replacement; validation and guardedness impose further conditions. A Lean model checks abstract reference closure, identity uniqueness and assignment enumeration. A full native refinement proof connecting these abstractions to Rust execution remains open, with established DPO mechanisation providing a relevant comparison (Söldner and Plump, 2024).

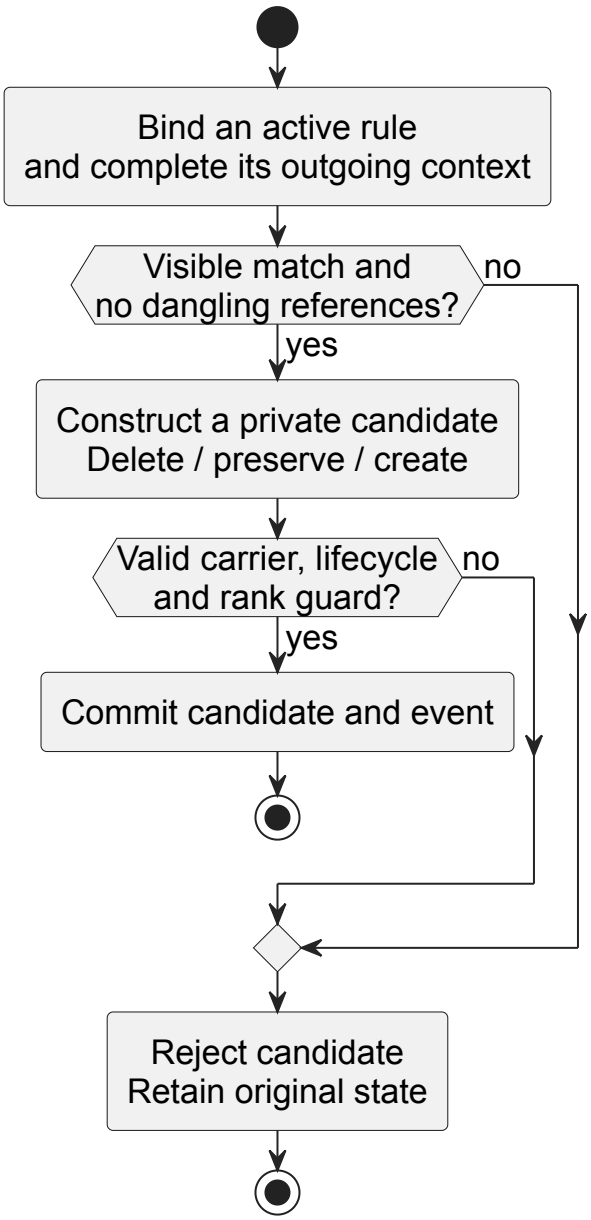



Figure 2. Ground replacement precedes transactional validation. Rejection preserves the original carrier.

## 3.4 Revisions, ranks and non-regress

A persistent rule head may acquire different executable descriptions. Revision identities distinguish them. Entry into the live set requires a fresh revision; exit consumes the old one. A continuing head receives a fresh revision exactly when BodyRep changes. Thus rank, mode, activation and syntax changes remain observable, while revision-only churn is rejected. The discipline includes inactive heads; deleting a head removes its representation.

For every entering or altered head q, guardedness GC requires:

$$\mathrm{rank}_{\mathrm{post}}(q) < \mathrm{rank}_{\mathrm{pre}}(p).$$

A rule can modify itself by lowering its rank. A higher-ranked supervisor can subsequently restore that rank below its own, allowing further modification.

Write $e \prec_v e'$ when $e$ enters or alters $q$, producing revision $\rho$, and later $e'$ applies $q$ using exactly $\rho$.

**Lemma 3.3 (revision continuity).** Along this link, $q$'s rank at $e'$ equals its rank immediately after $e$.

*Proof.* Any intervening alteration or exit consumes $\rho$; re-entry requires a fresh token. Since $e'$ reads $\rho$, neither occurred.

**Theorem 3.4 (exact-revision non-regress).** No admitted execution contains an infinite $\prec_v$-chain.

*Proof.* GC strictly decreases the applied rank across each link. Lemma 3.3 connects each post-rank to the next pre-rank. An infinite chain would be an infinite descending sequence of natural numbers.

**Corollary 3.5 (initial-rank bound).** If $M$ is the largest initial live rank, every chain has at most $M$ links, hence $M + 1$ events.

*Proof.* Unchanged heads retain their ranks; entering or altered heads have smaller ranks than an existing source. Induction bounds every live rank by $M$. Each chain then descends strictly.

The distinction between a rule and its revision is consequential. Without GC, peers swapping ranks one and zero can form an infinite revision chain. With GC, a rank-two supervisor can repeatedly restore lower-ranked rules before they execute, replacing their revisions. This continuing activity satisfies the theorem (Figure 3). Some safe equal-rank edits are rejected, motivating stronger measures for broader classes.

Stronger edit bounds require additional structure. In a finite complete carrier quotient, assign rule-changing edges cost one and others zero. Total edits are bounded exactly when no cycle has positive cost: removing zero-cost cycles leaves a bounded simple-path cost (Endrullis, Waldmann and Zantema, 2006). Some growing graphs admit such quotients by erasing isolated constructors whose labels and sorts no reachable pattern reads and no observer selects; delay-producing self-loops remain (Cousot and Cousot, 1977; Might and Shivers, 2006). Incomplete exploration is inconclusive.

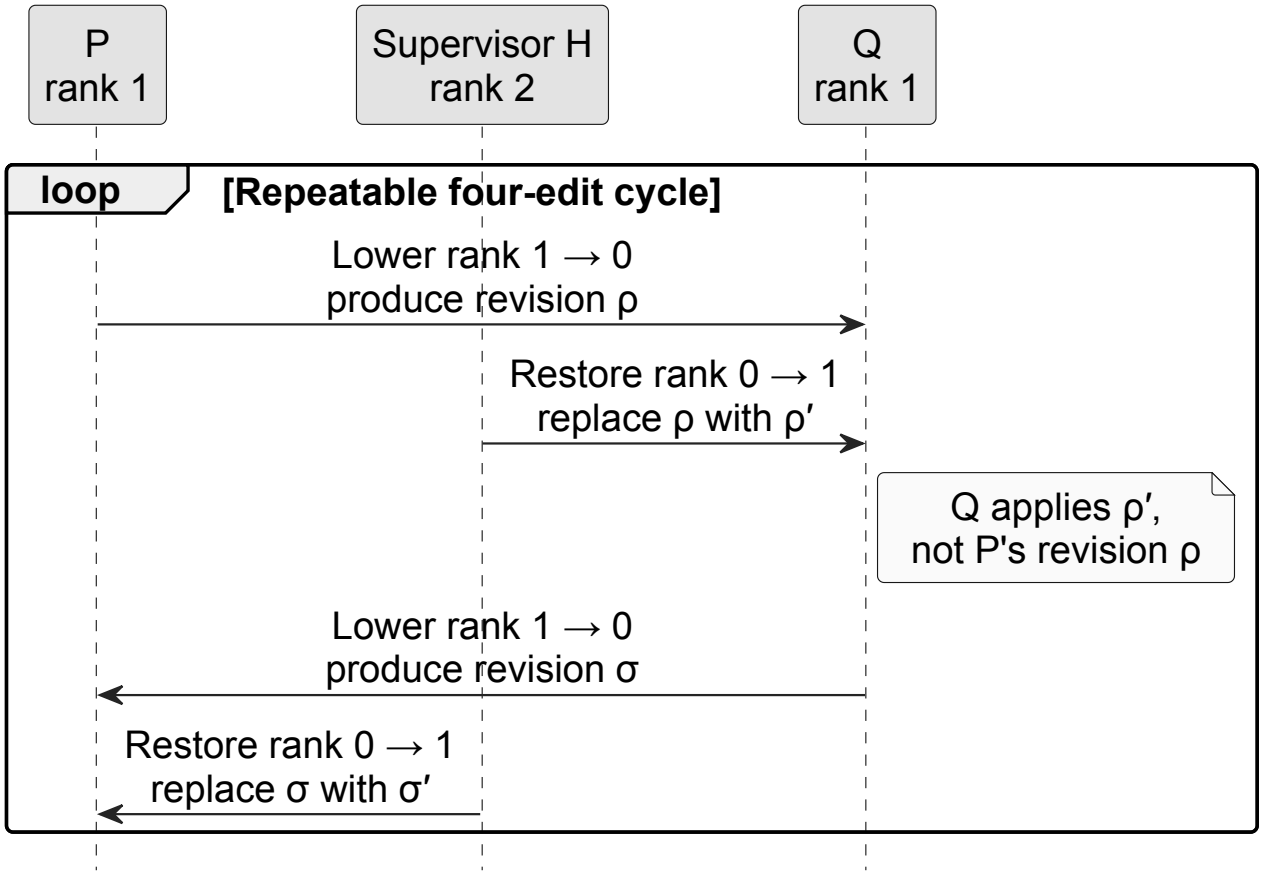



Figure 3. Supervised restoration permits continuing modification. Each arrow is an edit; restoration replaces the revision produced by the preceding lower-ranked rule.

## 3.5 The meaning of reduction

“Minimal” asks how much representational machinery the same capabilities require. The project compared encodings of four identical programs: arithmetic, installation, self-lowering and mutual rule revision (E1). Table 2 counts declared vocabulary/conventions and, separately, nodes/incidences in each encoded program. Direct edge-on-edge representation was examined but falls outside the ordinary-carrier class supporting Proposition 3.2. This is a scoped engineering comparison.

Table 2. Representation costs

| Encoding | Profile items | Arithmetic nodes/incidence | Installation | Self-lowering | Mutual swap |
|---|---|---|---|---|---|
| Separate records R | 23 | 31/39 | 33/47 | 23/31 | 81/117 |
| Generic records G | 18 | 31/39 | 33/47 | 23/31 | 81/117 |
| Default-Free G_F | 17 | 31/39 | 33/47 | 23/31 | 81/117 |
| Multiplicity universal U_M | 3 | 89/189 | 99/220 | 73/155 | 217/522 |
| Tagged universal U_T | 3 | 88/167 | 98/193 | 73/139 | 219/462 |
| Positional universal U_P | 3 | 65/105 | 69/117 | 52/87 | 150/276 |

Combining record categories into roles reduces R to G. Defaulting omitted bindings to Free yields G_F, which dominates G on this vector. The implementation retains explicit declarations for diagnostic clarity. Small universal signatures have precedent in interaction combinators (Lafont, 1997); here, structural encodings reduce vocabulary while enlarging graphs. Their decoding and evaluation costs remain relevant (Peters and van Glabbeek, 2015).

A second experiment exhaustively enumerated 238,480 small rooted graph gadgets to investigate the cost of representing an ordered pair (E1). For $i$ internal nodes, $k$ opaque sinks, $e$ arcs and outdegree at most $d$, connectivity gives $i + k - 1 \leq e \leq di$. Unary incidence cannot reach two sinks. An unlabelled simple ordered-pair gadget needs two internal nodes and three arcs; labels or multiplicities defeat unrestricted extensions. Lean (de Moura and Ullrich, 2021) checks the conditional arithmetic.

The executable comparison also translated admission queries between encodings and compared their outcomes, covering 4,095 nonempty sort subsets and 1,431 translated queries (E1). Multiplicity encoding $U_M$ is smaller than positional $U_P$ on some variable-heavy carriers. The alternatives therefore expose different costs rather than a universally best encoding.

Description complexity (Kolmogorov, 1965) also depends on observation. A finite summary that uniformly implements an interface and is recoverable from every correct deployment gives complexity $K(\text{summary}) + O(1)$ on optimal description references (Shen, 2015). Decoder costs remain included. Charging logarithmic initialisation work permits bounded exact search on a fixed effective reference (Allender, 2001). Fourteen Lean lemmas support these conditional bounds, without establishing global minimality or verifying the evaluator.

# 4. Semantic stability and convergence

Controlling self-reference leaves a second question: whether the resulting computation settles into a useful state. A graph may stop changing, retain a stable output while its rules continue changing, or improve particular answers as new work arrives. Each possibility requires an appropriate observation and progress argument.

## 4.1 Observations and fixed points

Let $\Omega$ contain valid configurations up to renaming that preserves types, attributes and field positions. Transitions respect this equivalence. An observer selects either the whole carrier or chosen atoms with their outgoing closure, comparing the resulting structures up to such renaming.

Strong stability means every successor within the declared domain preserves the observation. Eventual stability means sufficiently late observations agree. A normal form has no enabled rewrite, whereas a stable observation may coexist with continuing activity. Oscillation repeatedly visits distinct observation classes; unbounded growth offers another form of divergence.

For $X \subseteq \Omega$, let $\mathrm{Post}(X)$ be its successors. $F(X) = X \cup \mathrm{Post}(X)$ is monotone; its fixed points are forward-closed sets (Tarski, 1955). Adding initial set $S$ gives $F_S(X) = S \cup X \cup \mathrm{Post}(X)$, whose least fixed point is $\mathrm{Reach}(S)$ by induction on finite paths. This is reachability closure, not convergence to a meaningful state.

## 4.2 A sufficient normalisation class

Consider a fixed program with one active rank-zero object rule replacing $\mathrm{pending}(x, y)$ by $\mathrm{done}(x, y)$, preserving distinct constructor endpoints. Data contain only constructors and object links outside the rule representation; parallel links and shared endpoints are allowed. Each step removes one pending record and creates none. Every remaining pending record has an admitted match, so $k$ initial records require exactly $k$ steps in an unrestricted maximal execution. Different replacements commute, giving local confluence. Newman's theorem then yields a unique normal form. This supplies a reference case for examining reflective interference.

## 4.3 Stability without collapse

For the behavioural examples, useful content is a value-labelled link between distinct endpoints. Processing a pending link while preserving that content reaches a useful normal form. Erasing the value link also normalises, but loses the chosen payload. Alternating pending/done rules instead permit indefinite activity with the same payload preserved. With unlimited fresh identities, exactly one phase remains enabled at each step. This supplies a constructive distinction between completion, collapse and persistent modulation; an observed repeated display alone would not establish infinite execution.

## 4.4 Recovery with changing policies and bodies

Recovery can preserve useful observations while computation continues (Dijkstra, 1974). The analyser explores all admitted transitions modulo exact full-carrier isomorphism. A contract fixes payload identities, status relationships and a legitimate region $L$. Renaming preserves admission

and extends to fresh atoms, justifying quotienting (Rensink, 2007). Payload-only equality is insufficient: a removed quoted guard can change enablement without changing data.

For a finite complete quotient, require payload preservation, closed $L$, successor existence outside $L$ and acyclicity outside $L$. Put $V = 0$ on $L$ and otherwise $V = 1 + \max V(\text{successor})$. Every outside step decreases $V$, and no unfinished maximal execution deadlocks. Hence $L$ is reached within $V(\text{initial})$ steps without fairness. Stable outputs additionally require preservation on $L$-transitions. Exhausted exploration is inconclusive; finite allocation limits are outside the theorem.

Body-changing examples expose the assumptions. Removing one guard while processing $n$ pending items decreases $V = p + \text{guard}$, giving $n + 1$ steps. Restricting modulation to $L$ preserves recovery; unrestricted modulation can starve work. Revision may also change a terminal label from done to audited. Agreement across executions therefore requires a separate comparison of all reachable legitimate outputs (Plump, 2010).

A stronger family has $n$ independent payloads with pending/audited/done statuses. A guarded meta-rule changes pending → done into pending → audited; a fixed audited → done rule completes processing. The guard prevents editing that normaliser. $W = 2p + a + b$, counting pending items, audited items and remaining upgrades, decreases on every step; all nonterminal components have successors. Thus every execution finishes within $3n$ steps with identical done outputs (Figure 4). Removing the guard admits a loop. Finite faults within this component domain recharge $W$; telescoping bounds rewrites by initial $W$ plus positive fault increments. Recovery follows after the last fault.

Continuing inputs introduce a service requirement: arriving work must remain within processing capacity for backlog to remain bounded. Individual waiting times additionally depend on fairness. Structural damage raises a different problem. External reconstruction repairs selected missing or duplicate statuses while preserving payloads and rules; unsafe incidence and lost payloads are rejected. Such repair requires an explicit consistency contract (Sandmann and Habel, 2019; Fritsche et al., 2026).

## 4.5 Structural contracts and persistent work

A sufficient checker recognises task components containing a persistent ticket, immutable payload, identity link and unique status. An external contract fixes natural status weights $w$, zero-weight terminals and permitted routes. Rank-zero object processors replace statuses and may create children. Guarded rank-one supervisors replace processor RHS statuses and revisions while preserving inputs, activity and children. The class must remain closed under every such replacement and supply a processor for each nonterminal status. Extra structure and shared rule syntax are rejected.

For each route $a \to b$ with child stages $C$, require $w(a) > w(b) + \sum_{c \in C} w(c)$. Summed task weight $W$ then decreases on processing and is preserved by administration. Visibility excludes syntax matches; component closure excludes dangling deletion; supervisor guards preserve the processor class. Thus unfinished work always has an admitted processing step, and terminals cannot be processed.

Consuming one nonrenewable permit per edit gives an all-orders bound $W + P$. Unlimited administration instead needs fairness over persistent actions (Lamport, 2000). The FIFO scheduler retains ticket identities across revisions, appends children behind waiting tasks and permits $B$

administrative steps between processing steps. With remaining credit $c \leq B$, $\Phi = \max(0, (B+1)W + c - B)$ decreases until completion: administration reduces $c$; processing reduces $W$ and resets $c = B$. Completion follows within $(B+1)W_{\text{initial}}$ commits.

Lean verifies the abstract arithmetic and fair-recovery argument; native recognition remains supported by tests. The project's routing/branching experiment varied task count, permitted administrative burst and finite versus persistent revision, producing forty-eight cases. All completed within their bounds; their 1,058 transitions were replayed through native admission (E2). The contracts exclude arbitrary corruption and continuing arrivals, and allow disagreement when different terminal choices are permitted.

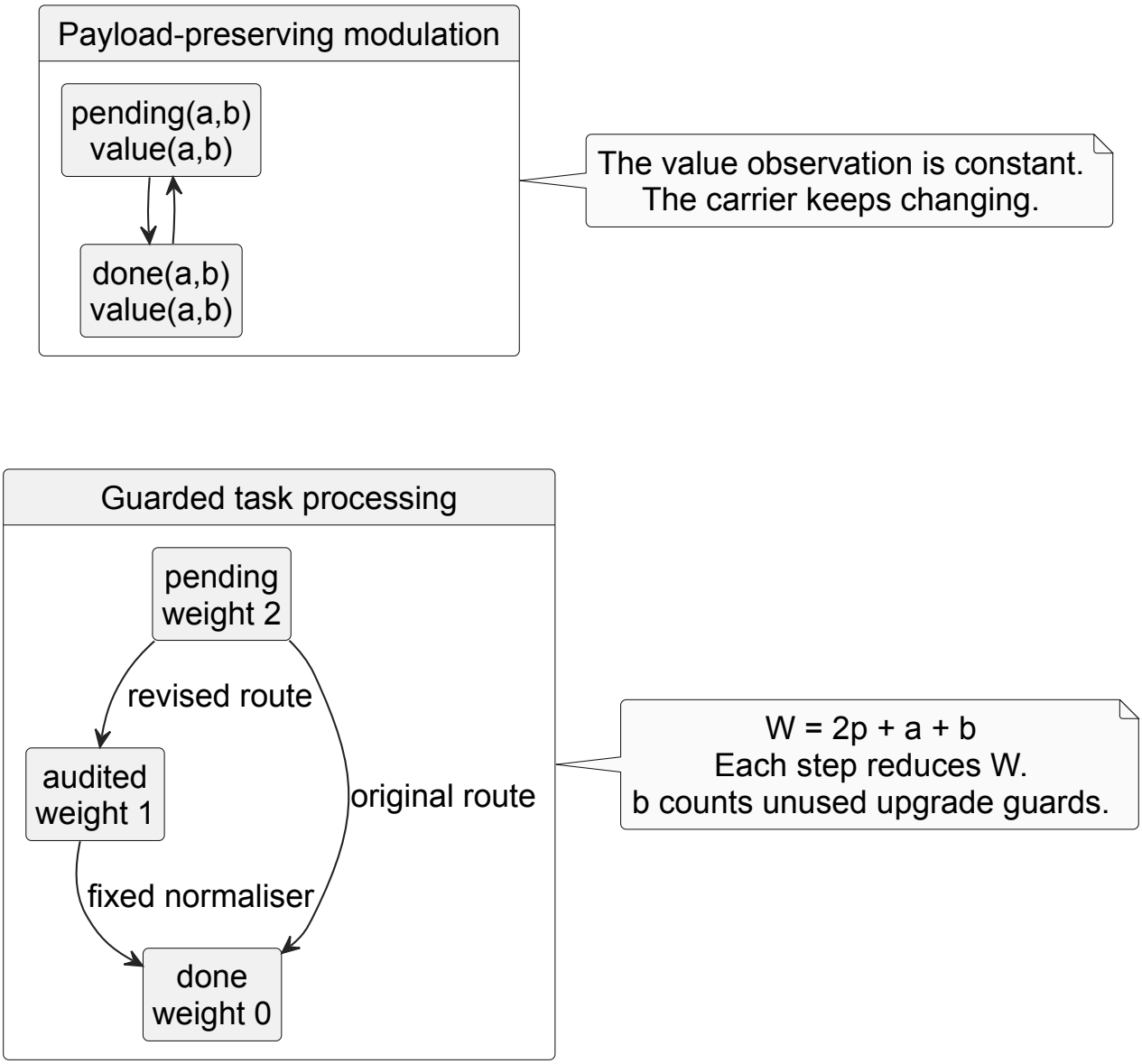



Figure 4. Two program families: payload-preserving oscillation (upper) and guarded normalisation (lower). The lower routes show alternatives before and after a permitted rule revision.

### 4.6 Local numerical accuracy during continuing evolution

Established tasks may become accurate while new tasks arrive. For persistent identity $i$, local convergence means $x_i(t) \to v_i$, its fixed solution; maximum error over all arriving tasks need not vanish. Let $H\colon [0,1]^{\mathbb{N}} \to [0,1]^{\mathbb{N}}$ be monotone and product-continuous, with $H^n(\mathbf{1})_i \to 0$. If service, fresh reads and disturbances imply $e \le H(e)$ for componentwise limsup error $e$, induction gives $e \le H^n(\mathbf{1})$, hence $e = 0$. Establishing that inequality remains application-specific.

To make this distinction executable, the project includes a retry/delegation application: tasks may retry locally or delegate work, and their equations determine eventual success probabilities. Its Rust solver uses the discrepancy in those equations to bound present error, accounting for input changes and rounding (Rump, 2010). The task inputs, numerical outputs and independent comparisons are retained in E3. Finite-step stability has established theory (Geiselhart and Wirth, 2016). Accuracy throughout a cycle requires bounded intermediate departures: $x := 1;\ x := 0$ has stable cycle endpoints but repeated internal variation. Rust performs this numerical calculation and publishes checked results through a graph rewrite. The application illustrates why observation and service assumptions matter when useful computation continues.

# 5. Event dependencies and temporal structure

A rewrite producing information needed by another establishes a dependency. This ordering can differ from the scheduler's chronology.

### 5.1 Resource order

A committed event records its source revision, binding, consumed atoms $C$, preserved context $K$, produced atoms $P$ and reads $R$. Reads include the completed match and applied rule representation, including consumed items. Immutable owned fields make outgoing structure part of each read.

For events $i < j$ in one history, insert a direct dependency when either:

- $P_i \cap (R_j \cup C_j) \neq \varnothing$; or
- $R_i \cap C_j \neq \varnothing$.

These conditions connect production to use and reads to later deletion; shared reads add no edge. Every dependency points to a later event, so transitive closure gives a strict partial order. Tracking each atom's producer and readers recovers the stated intersections inductively.

### 5.2 Scope of the resource order

A meta-event may remove quotation and expose a constructor without producing anything the later reader uses. The resource order misses this enabling condition. Tracking removed initial blockers explains visibility under monotone removal, but broader cases require comparing alternative histories.

The intervention analysis therefore enumerates order-preserving subsets of earlier events. Each retained occurrence keeps its source revision and binding and must pass admission, with fresh identities transported consistently. Infeasible alternatives are distinguished from exhausted

analysis.

Queries test either the exact occurrence or any binding for its source revision. Only the latter rematches. Reports retain outcomes, minimal successful selections and their intersection. Conditional on native correctness and complete matching, induction establishes replay feasibility and exhaustive enumeration establishes completeness within this finite intervention universe. The argument is manual.

Supports are not upward-closed: quotation can block a reader, while alternative producers can support source-level admission. The project's independent Boolean oracle verifies 6,132 outcomes across eighteen designed cases and three repetitions (E4); tests also cover completed context, dangling and exhaustion. Enumeration is exponential, with default/hard predecessor bounds eight/twelve.

Local interference checks both residual orders. If both remain admitted, the program is unchanged and consumption avoids the other's completed reads, pairing fresh atoms by occurrence/template gives a carrier isomorphism. Distinct-link normalisation satisfies this sufficient criterion.

Negative-condition complementation has restricted hypotheses (Corradini, Ghaffari Saadat and Heckel, 2020); signed provenance addresses negation (Riddle, Köhler and Ludäscher, 2014). Neither general independence nor Halpern's (2015) structural-model actual causation follows.

## 5.3 Tracing selected influences

For a long execution, a developer may want to follow only particular changes, such as a policy revision and the result it influenced. Selected provenance attaches labels to those events and carries the labels through intermediate computations. An unselected step can therefore transmit an earlier influence. This explains the ancestry of the observed result; necessity requires examining alternatives.

For event labels $\tau(e)$, initially set atom support to empty. An event's ancestry unions support over actual reads; fresh atoms receive that union plus $\tau(e)$ (Figure 5). Union-image commutes with both operations: induction proves equality with projecting full birth/read ancestry, including many-to-one labels. Creator, ancestry and earlier history differ: unselected intermediaries transmit ancestry; readers producing nothing add only read/consume ordering constraints.

Native analysis recomputes ancestry from verified histories (E4). Necessity across alternatives additionally requires a nonempty complete context family. A compressed interface preserves observation paths if every concrete step projects and every interface step lifts from every corresponding concrete state. Seven Lean statements check these obligations, without verifying Rust. This conditional projection is related to provenance semantics (Grädel and Tannen, 2017), but weaker than hereditary history-preserving folding (Baldan and Raffaetà, 2019).

## 5.4 Checked histories

Histories retain the initial carrier and enough information to replay each admitted occurrence. Checking the source revision, binding, allocation and effects reconstructs the execution inductively. Reproducing the scheduler's choices additionally requires its ordering, seed, limits and observer. This distinction supports both inspection of an existing history and controlled repetition of the computation that produced it.

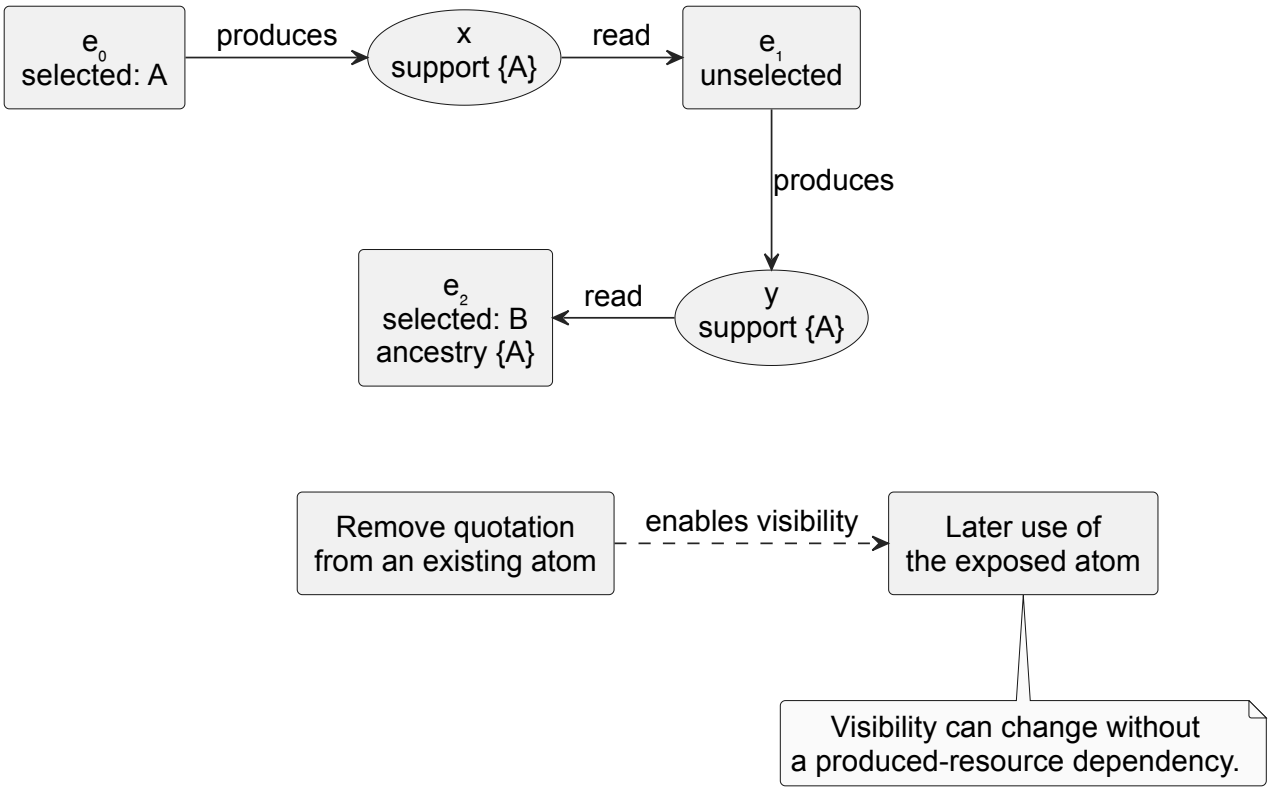



Figure 5. Events (rectangles) transmit selected ancestry through produced and read atoms (ellipses). The lower example shows a visibility influence absent from that resource path.

# 6. Expressiveness

A general computational example is needed to establish that resident rewriting can do more than update a few statuses. The project therefore compiles lambda expressions—functions and applications—into graphs and evaluates them using ordinary resident rules (Figure 6). The correspondence result connects each graph step to a recognised source-language meaning.

## 6.1 Source and graph machine

The source language is finite pure untyped lambda calculus: $M ::= \mathrm{Var}(n) \mid \mathrm{Lam}(M) \mid \mathrm{App}(M, M)$. Terms may be closed or use an ordered context $\Gamma$ of free names. Indices address enclosing binders and then $\Gamma$. Weak-head call-by-name reduces applications until reaching a function or a neutral expression headed by a free name; function bodies and neutral arguments stay unevaluated (Nogueira and García-Pérez, 2024). Constructors and links encode acyclic syntax, environments, argument stacks and one machine-control state.

A machine state is $(t, e, s)$, where a closure pairs term and environment, and the stack stores pending argument closures. Four schemes implement evaluation:

Table 3. Closure-machine transitions

| Scheme | Before | After |
|---|---|---|
| Push | $(\mathrm{App}(t,u),e,s)$ | $(t,e,(u,e)::s)$ |
| Beta | $(\mathrm{Lam}(b),e,(u,f)::s)$ | $(b,(u,f)::e,s)$ |
| LookupZero | $(\mathrm{Var}(0),(u,f)::e,s)$ | $(u,f,s)$ |
| LookupSucc | $(\mathrm{Var}(n+1),(u,f)::e,s)$ | $(\mathrm{Var}(n),e,s)$ |

Open compilation supplies $\mathrm{Free}(x)$ closures for $\Gamma$. Because graph matching is injective, the four schemes require eleven specialised rules covering possible shared references. Push and Beta each add four atoms; obsolete cells remain stored, making memory growth a practical limitation.

## 6.2 Invariant and correspondence

The invariant requires a fixed disjoint program, unique control, acyclic immutable syntax and cells, and correctly scoped closures. Environments point only to older environments and tails; execution begins with $\Gamma$ and an empty stack.

Readback substitutes environment-addressed indices with captured closures and reapplies stack arguments. Insertion beneath $d$ binders shifts free indices by $d$, preserving internal bindings. Thus $(\lambda z.\lambda x.z)\,x$ yields $\lambda.x$ with free $x$. A separate insertion offset prevents capture.

**Lemma 6.1 (preservation).** Every encoded state satisfies the invariant, and every machine step preserves it.

*Proof.* Initial scoping follows by structural induction. Push captures a scoped argument, Beta extends its environment and lookup selects an existing entry. Fresh cells point backwards in allocation age; program, syntax and unique control are preserved.

**Lemma 6.2 (exact applicability).** An invariant state has exactly one enabled occurrence unless it is a lambda with empty stack or Free with any stack. Both are terminal.

*Proof.* The syntax constructor selects a scheme, scoping supplies its entries and the disjoint sharing cases select one injective pattern. The invariant ensures admissible replacement. Free and an unapplied lambda have no transition.

**Lemma 6.3 (reflection and administrative progress).** Push and lookup preserve readback; Beta produces one weak-head beta reduction. There is no infinite administrative-only segment.

*Proof.* Readback identifies Push with deferred application, lookup with environment substitution and Beta with capture-avoiding substitution. Administrative steps decrease the pair (environment age, syntax height) lexicographically: Push lowers height and lookup moves to an older environment.

**Theorem 6.4 (weak-head correspondence).** With unlimited resources and no external interference, encoded evaluation simulates every source weak-head reduction by finitely many administrative steps followed by Beta; every target step reflects either equality or one such source reduction. Source and target weak-head termination agree.

*Proof.* Lemmas 6.2–6.3 give a finite administrative prefix for each source reduction. Induction supplies simulation; readback supplies per-step reflection. An infinite target computation contains infinitely many Beta steps, while terminal readback is weak-head irreducible.

The result applies to encoded states satisfying the invariant. External editing and software resource limits require separate checks.

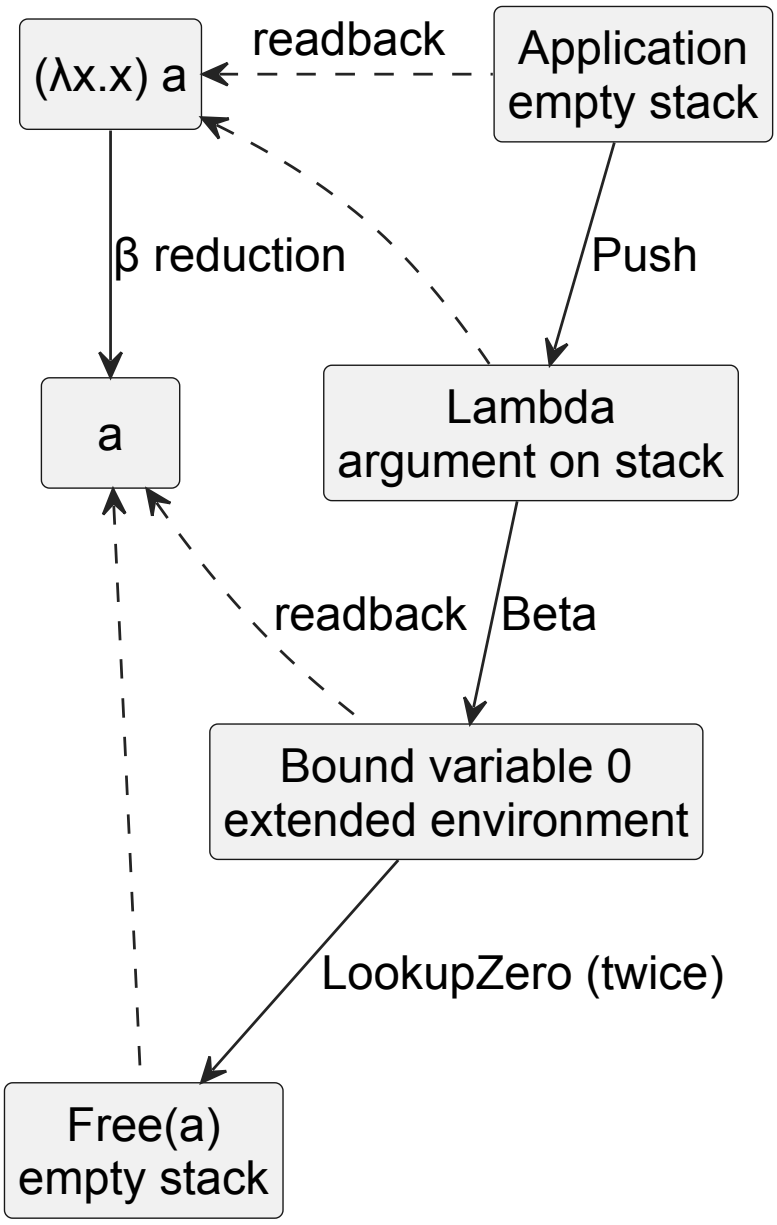



Figure 6. Readback relates graph-machine configurations to source expressions. Solid arrows are transitions; dashed arrows are readback. Push and lookup preserve the expression, while Beta reduces it. The final machine arrow abbreviates two lookups.

## 6.3 Ranked term graphs

**Proposition 6.5 (ranked incidence).** Encode symbol-labelled vertices as constructors and each ordered successor at position *i* as a distinct arg: *i* object link. Retain an external root and require exactly the prescribed positions. Labels and positions reconstruct the heap up to renaming; induction over finite successor paths preserves rooted unfolding, including sharing and cycles.

Root-preserving injective ground edits become rank-zero object rules when they preserve labels, replace changed links explicitly and account for all deleted incidence. Proposition 3.2 supplies the replacement correspondence within this encoding. Non-injective matching, implicit redirection and infinitary convergence require further treatment (Bahr, 2012).

### 6.4 Version-aware interpretation

A second interpreter examines update continuity (Stoyle et al., 2007) within checked reflection (Amin, 2026). Return and DelayReturn preserve an argument's identity in one or two rewrites. Seven rules publish a budget-checked implementation and let each call retain its captured version. Readback checks installed bodies, while abstract proofs establish value preservation and the replacement's evaluation bound. Older calls retain their original behaviour.

## 7. Software design and implementation

The software turns the formal model into a developer platform for writing graph programs, running them locally or as hosted jobs, and explaining their execution. A user can start with the guided example, inspect a changed rule, save a source version and recover its recorded history.

### 7.1 Architecture and semantic authority

Five Rust crates separate responsibilities. Graph-core stores and validates the carrier; engine matches and admits rewrites, schedules execution and records history; parser translates authored programs; CLI and wasm-bridge expose these capabilities to native processes and the browser. Both interfaces use the same engine (Figure 7).

Atoms are immutable typed records indexed by never-reused 64-bit identities. An ordered persistent map shares unchanged storage between states; a reverse-incidence index locates incoming references. This supports candidate construction without copying the complete carrier and efficient checks against dangling references (Driscoll et al., 1989). Only a validated candidate can become the next public state. Revisions also invalidate stale prepared changes.

React and TypeScript implement the editor and linked views. A Web Worker hosts the WebAssembly engine, keeping computation off the interface thread; typed messages carry commands, diagnostics and bounded results. Source spans connect validation errors to editor locations. Browser layout and filtering operate on projections of the recorded carrier, leaving execution unchanged.

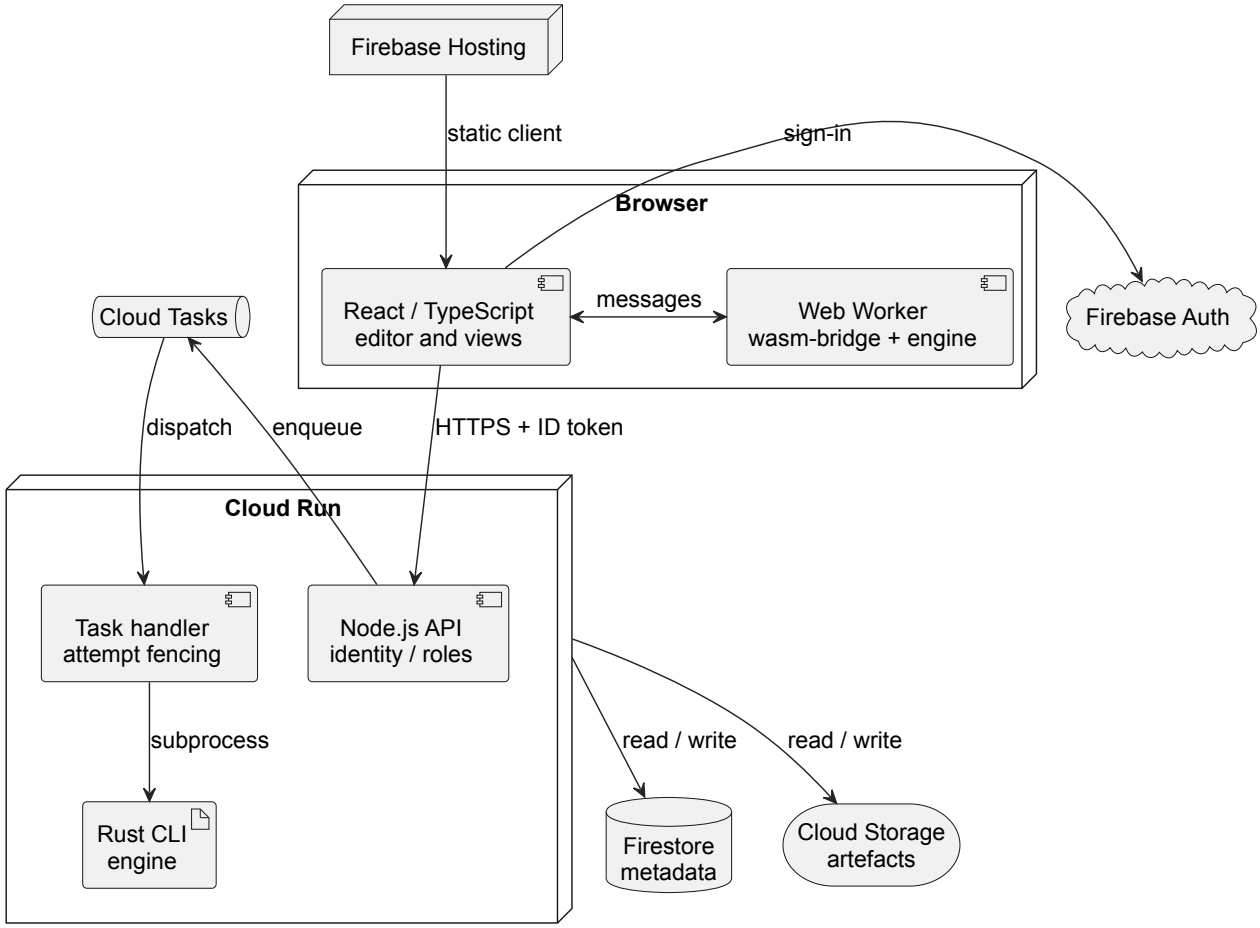



Figure 7. UML deployment view of the platform. Browser and hosted execution share the Rust engine. Components exchange typed messages, authenticated requests and stored artefacts across the indicated boundaries.

## 7.2 Execution and scaling

General execution searches for admitted occurrences before choosing a step. Seeded selection makes fixed settings repeatable; resource exhaustion is reported as inconclusive. This search becomes costly on larger graphs. Following incremental-matching approaches (Varró, Varró and Schürr, 2006; Beyhl and Giese, 2016), specialised profiles reuse matches for status changes and functional joins whose effects preserve the necessary conditions. Other programs retain general admission. Recurrence detection similarly uses inexpensive screening followed by exact graph comparison (McKay and Piperno, 2014).

## 7.3 Authoring and inspection

Editing, diagnostics and breakpoints support program development. Four views present practical relationships (Figure 8), internal storage, rules and history. A timeline reconstructs earlier states, allowing the reader to connect a visible change with its applied rule revision. A single-program walkthrough introduces these relationships. Selective labels and graph navigation support inspection as the carrier grows.

## 7.4 Developer platform

The hosted application serves the React client through Firebase Hosting. Email-verified users access persistent projects, immutable source versions, role-based sharing and recoverable jobs through a versioned HTTP API. A Node.js service on Cloud Run verifies Firebase identity and project permissions. Firestore stores project and job metadata; Cloud Storage holds source and trace artefacts (Firebase, n.d.).

Submitting a job records its settings and source version before Cloud Tasks dispatches bounded execution to a native Rust subprocess. Jobs publish checked checkpoints so interrupted work can resume. Attempt identifiers prevent an obsolete worker from replacing an accepted result, while idempotency keys prevent duplicate requests from creating duplicate jobs. These boundaries preserve the same admitted execution semantics across local and hosted use.

Local WebAssembly execution remains available. Adaptive workflow requests are ephemeral, while stored jobs provide durable execution. Numerical and learning applications share the Rust infrastructure but require their own checks for calculations and fitting preceding graph execution. The hosted pilot demonstrates these workflows at assessment scale; industrial availability and external effects would require further evaluation.

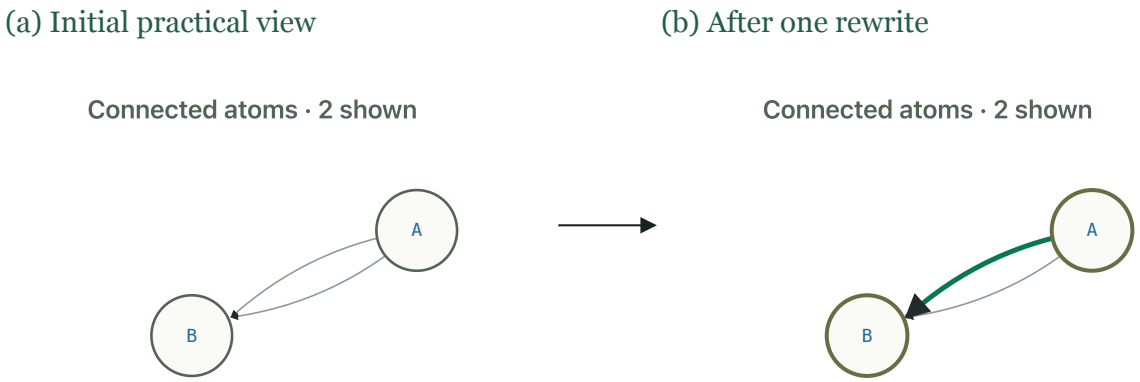



Figure 8. Practical graph views before and after one rewrite of the payload-preserving example. The thicker green relation is the new done record; the value record and endpoints persist.

## 7.5 Development process

Sprint 1 began with a literature review from 1 June to 29 August 2026, followed by formal foundations from 29 August to 6 September. Development then progressed to executable semantics, browser inspection and hosted applications (Figure 9). Counterexamples revised the rank discipline, and full-run measurements prompted changes to trace validation. Sprint 6 combined platform development with further research and integration. The recorded project period ends on 11 October 2026; subsequent editorial work does not extend the implementation timeline.

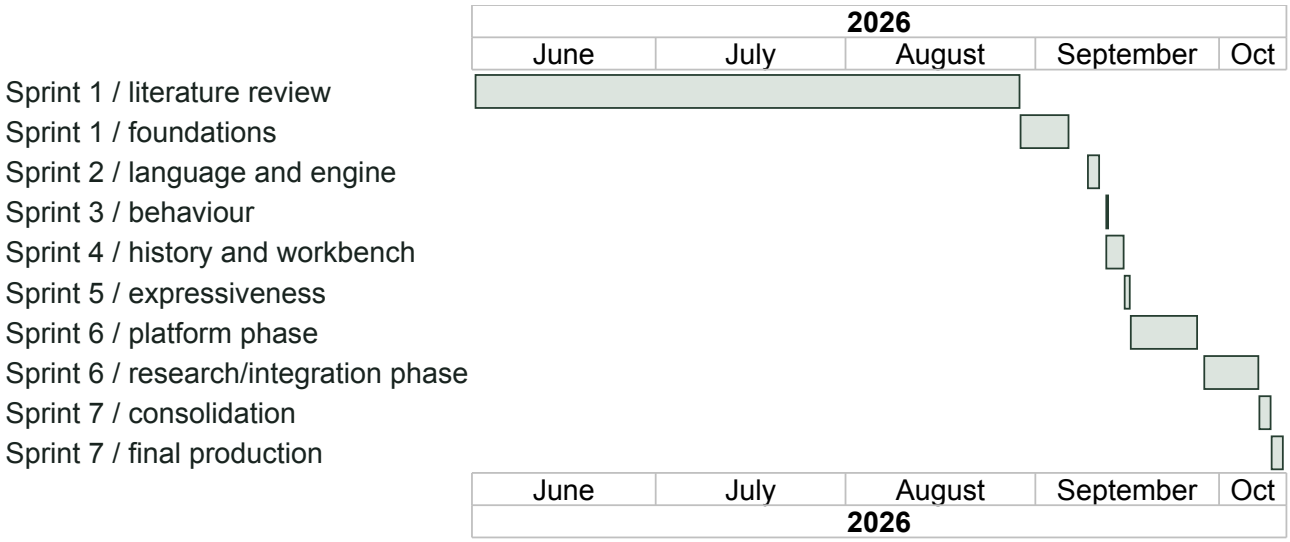



Figure 9. Development timeline, 1 June–11 October 2026. Sprint 1 comprises literature review and formal foundations; Sprint 6 comprises platform development and research/integration.

# 8. Evaluation

Table 4 links empirical claims to inputs, captured results and checks at a fixed public revision. Historical manifests identify the measured implementation. Inspection requires no rerun; reproduction uses the repository's toolchain and linked experiment drivers.

Table 4. Experimental evidence and reproduction

| Record | Experiment and scope | Inspectable evidence |
|---|---|---|
| E1 | Representation: four program encodings; 69 snapshots, 1,431 translated queries; 47 storage snapshots. | Encoding report; storage report; verifier |
| E2 | Workflow recovery: 48 routing/branching cases; 1,058 replayed transitions. | Inputs, outputs and manifest; live replay verifier |
| E3 | Numerical application: 80 tasks × 3 solvers; 3,000 coordinate checks. | Rust/reference comparison; experiment driver |
| E4 | Dependencies: 18 intervention cases × 3 repetitions; 6,132 outcomes. Selected provenance: 24 histories, 450 projections. | Intervention runs; provenance report; verifier |
| E5 | Conformance: generated campaigns and 19 catalogue programs / 23 execution profiles. | Seeds and test commands; catalogue settings; expected results; verifier |
| E6 | Version-aware execution: 32 fixtures, 880 terminal paths and 2,684 edges. | Report, captured output and reproduction commands |
| E7 | Lambda correspondence: 450 context-scoped source trees; 704 graph rewrites. | Captured runs; enumeration specification; live verifier |
| E8 | Native/WebAssembly agreement and reproducible builds. | Cross-environment report; verifier |
| E9 | Matching and complete execution: 100–100,000 items, time and peak memory. | Candidate-memory study; historical runs; initial repeat; corrected repeat; verifier |

| Record | Experiment and scope | Inspectable evidence |
|---|---|---|
| E10 | Public-data transformation: 12,330 rows; conventional and graph processing. | Dataset protocol and commands; original results; corrected results |
| E11 | Supplied and acquired interpretation: calendar/fiscal/non-calendar tasks and controls. | Interpretation evidence; policy-learning evidence |
| E12 | Convention maintenance: 191 conditions; native replay and compiled execution. | Condition analysis; native report; validation report; reference verifier |
| E13 | Action acquisition: 7,776 observation prefixes; 324 native executions. | Commands and captured evidence; experiment driver |

For example, after building the CLI with `cargo build --release -p cli --locked`, `node scripts/workflow-study.mjs verify benchmarks/results/workflow-contracts-01 --live` rechecks all workflow cases against native admission. Frozen-result verification and fresh execution are distinct checks; hardware-dependent timing is expected to vary.

## 8.1 Conformance with the formal account

Evaluation examined whether software execution agreed with the definitions and whether the resulting behaviours supported the research questions. Eight generated property campaigns supplement hand-derived examples; their repeatable seed/count profiles are retained with the test source (E5). Nineteen catalogue programs cover twenty-three native/WebAssembly profiles. Version-aware tests examined 32 fixtures, 880 terminal paths and 2,684 transition-tree edges (E6).

Self-lowering and supervised restoration demonstrate the distinction between bounded revision chains and continuing rule modification. Stable, oscillating and collapsed examples distinguish useful output from termination. Visibility counterexamples reveal omitted enabling conditions. Selected-provenance analysis agrees with independent dependency traversal across 24 histories, 90 events and 450 projections (E4), supporting the explanation of recorded executions.

An independent substitution evaluator agrees on 450 context-scoped trees and 704 rewrites, including capture and aliasing cases, within the stated weak-head strategy (E7).

Selected programs agree across native/WebAssembly execution on macOS and two Linux environments. Remapped same-target native builds reproduce bytewise; WebAssembly bytes differ across operating systems. Semantic agreement and executable identity are distinct findings (Lamb and Zacchiroli, 2021; E8).

## 8.2 Performance and useful interpretation

The project's performance study used an Apple M4 Pro with 48 GiB memory and separated processing, recording and replay; resident-set size (RSS) measures whole-process memory (E9). Retaining bindings rather than candidate graphs reduces thousand-candidate RSS from 261.75 to 4.63 MiB, but larger general queries still time out.

In the recorded study, restricted incremental execution reached 100,000 status changes with recurrence and verified history enabled. The median complete cost was 130.245 seconds and 3,478 MiB RSS; recording and verification dominated processing. This demonstrates a recognised fixed-

rule class, not arbitrary reflective programs.

Repeated measurement exposed excessive temporary allocation during trace validation, exceeding the 6-GiB ceiling. Streaming checks and comparison without copying subsequently completed the warm-up and all three measured 100,000-item attempts, with measured medians of 22.780 seconds and 5,858 MiB RSS (E9; Figure 10). Histories and resource limits were unchanged. The contrast identifies validation as a significant engineering cost; timing differences also depend on the measured environment.

A public-data application transforms all 12,330 shopping-session rows while preserving their eighteen source fields (Sakar and Kastro, 2018). Graph-resident approval activates a month-category transformation. The original median was 53.879 seconds, compared with 18.321 ms for conventional processing; the corrected fixed-policy runs had a median of 10.848 seconds (E10). Detailed execution records carry substantial overhead, making traceability a trade-off requiring justification for each application.

Task interpretation is evaluated separately from execution legality. Context-sensitive tables correctly distinguish calendar, fiscal and non-calendar readings, while wrong-table and rule-free controls can also reach normal form. Example-driven policy selection (Gulwani, 2011) ties a correct fixed baseline. These findings support task-relative interpretation, with the supplied conventions and observations remaining external (Harnad, 1990; E11).

Across eighty generated numerical tasks and three solver methods, all 240 Rust runs agree with the reference; 3,000 independent coordinate-bound comparisons support the error estimates (E3). High retry probabilities defeat bounded raw iteration while direct solving succeeds: sound uncertainty reporting does not guarantee timely accuracy.

Independent accounting agrees on 47 snapshots (E1). Two carriers with 4,096 atoms and fields occupy 277,327 and 406,351 bytes because their labels differ. Entity counts therefore understate storage costs, even before accounting for the decoder, evaluator and history.

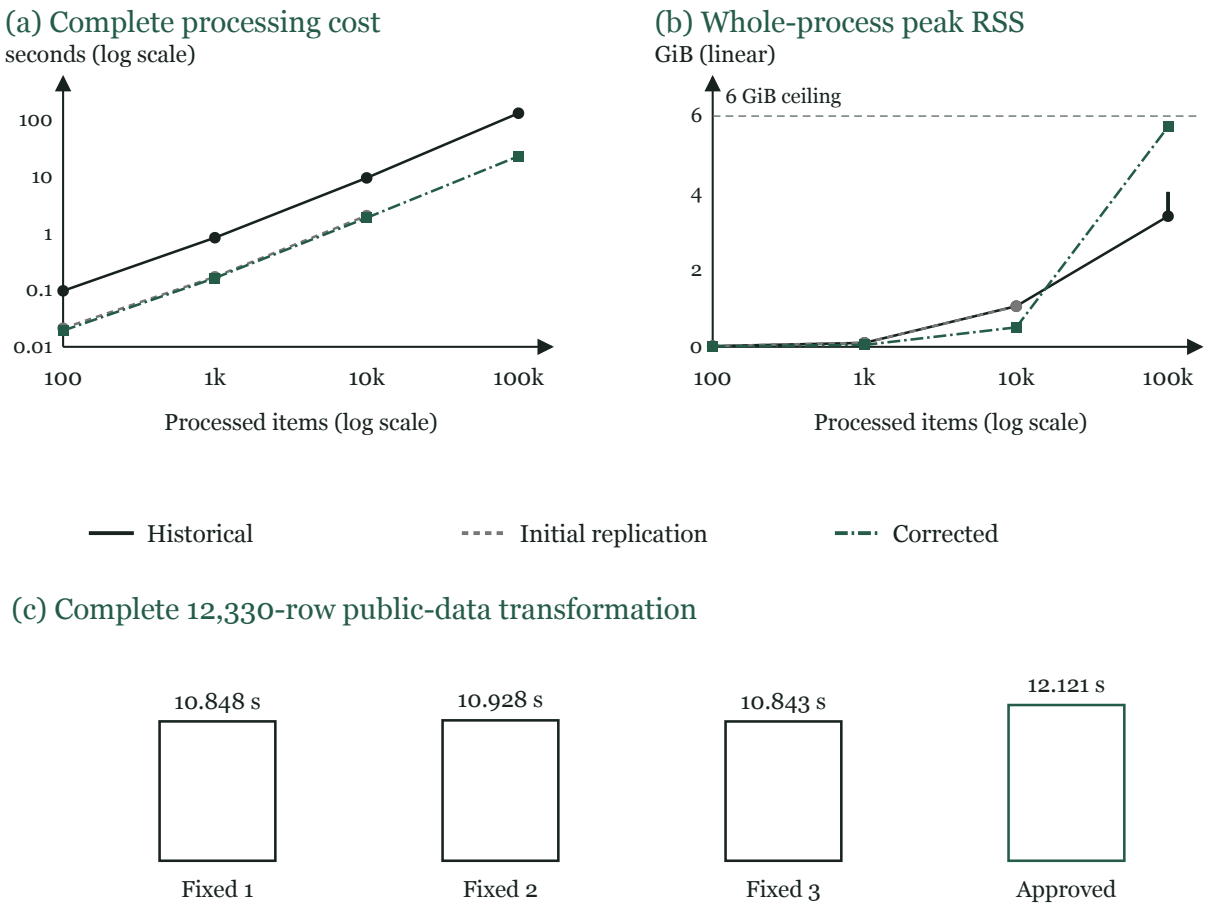



Figure 10. Complete processing time and peak memory (a–b): medians and ranges of three measured repetitions per completed cell. The initial repeat's four incomplete 100,000-item attempts have no completion estimate. Public-data transformation (c): three fixed-policy runs and one approved-policy control. Sources: E9–E10.

## 8.3 Acquired representations and renewal

The organization study examines whether acquired descriptions remain useful as the components interpreting them are replaced. Following studies of mutually supporting codes and interpreters (Füchslin and McCaskill, 2001; Wills, 2023), it couples transition records to navigation, resources, checking and inheritance. The experiment varied resource, checking and inheritance conditions across 191 conditions and eight diagnostic seed lineages (E12); four parents meet the combined functional criteria. Two descendants succeed with larger population transfer; none succeed with smaller transfer. Corrupting both checkers conceals errors, exposing the dependence on reliable observation. Rust reproduces reference states; compiled tables agree with a separate evaluator across 132 executions and 288 steps. Related runs and criterion-dependent ablations limit generalisation, but the results provide a concrete case of convention maintenance under RQ3.

Relational action acquisition intersects observed input literals into conservative preconditions and collects changed values as effects (Juba, Le and Stern, 2021). For stationary deterministic conjunction/assignment actions, true preconditions remain required; unobserved effects already hold in every permitted input. Predictions therefore have the true successor, including under injective typed renaming and a fixed frame. Sparse evidence can nevertheless prevent useful action.

Rust fitting is external; resident rules execute supplied action words. Contradictions trigger repair or quarantine, although changes can invalidate predictions before detection. Exhaustive enumeration of small deterministic action models checks 7,776 observation prefixes, 31,104 classifications and 9,000 safe predictions; 324 native executions make 144 rewrites (E13). This supports task-relative RQ3, without autonomous planning or intrinsic meaning.

A membership-evidence analysis, related to black-box checking (Peled, Vardi and Yannakakis, 1999), shows that existential path-language queries require accepted witnesses for positive answers and complete rejection for negatives. Removing an unwitnessed positive language or adding an untested negative word gives regular counterexamples. Finite membership evidence therefore cannot certify negatives with infinitely many path words without additional assumptions.

### 8.4 Threats to validity

Construct validity depends on declared costs, observations and causal scope. Shared development can correlate implementation and test errors; bounded independent oracles mitigate this risk without verifying Rust. Reused seeds, designed task families and composite success criteria limit statistical inference. One-machine timings and functional interface tests establish neither industrial scalability nor usability. Historical measurements identify their measured implementations, not final-build performance; participant and deployed-client studies remain absent.

## 9. Discussion and conclusion

### 9.1 Research outcomes and significance

The investigation establishes that a finite graph can contain executable descriptions of its own transformations while supporting precise reasoning about their effects. Progress, stable observation and dependency become inspectable properties of self-modification.

RQ1 yields a reduced representation and a bound on chains through exact rule versions. Comparisons reveal competing costs, leaving global minimality open. RQ2 establishes termination and recovery for identified program families, including selected changes to rule bodies. RQ3 shows why useful stability requires an observation criterion: an execution may finish incorrectly, erase its payload or preserve a useful result through continuing activity. Acquired representations add evidence that predictions can help sustain the processes using them, provided checking and environmental conditions remain suitable.

RQ4 derives a temporal partial order from recorded resource dependencies and strengthens explanation through selected provenance and bounded interventions. Absence and alternative histories remain important limits. RQ5 establishes a uniform correspondence with weak-head lambda evaluation, demonstrating computation beyond the individual examples.

Together, these findings refine the ambition of internal interpretation. More of the system's operational behaviour becomes represented and changeable within the graph. Intrinsic meaning remains unresolved: application labels and standards for useful output depend on supplied conventions and observations.

## 9.2 Practical significance

Consider a service whose routing policy changes while requests are still being processed. A useful system must distinguish the policy each request followed, preserve unfinished work and explain unexpected outcomes. Here, the policy is an addressable graph structure: its revision is recorded, supported updates can be checked, and the resulting execution can be replayed. The workflow and version-aware interpreters demonstrate restricted instances of these requirements.

This combination could support auditable data preparation, evolving business workflows and tools for investigating adaptive software. Its value would lie in making changes accountable to explicit contracts and inspectable evidence. The shopping-session study also exposes the price: retaining detailed histories is substantially more expensive than conventional processing. Adoption would therefore depend on whether explanation, controlled revision and recovery justify that cost. Industrial deployment and comparative usability remain further work.

## 9.3 Further research and conclusion

Broader convergence results require methods that admit more rule changes while retaining a sound measure of progress. Proof-carrying revisions (Lee and Necula, 1997) and size-change analysis (Lee, Jones and Ben-Amram, 2001) offer directions beyond finite exploration. More complete causal models would need to represent conflict and alternative executions. Global minimality requires stronger lower bounds that include decoding and evaluation costs. Work on acquired meaning should reduce the supplied interpretive machinery and test whether useful conventions survive unfamiliar environments.

Further software work should address general matching, memory reclamation and realistic domain workloads. The broader ambition remains compelling because useful software increasingly has to change while it runs. Placing procedures inside the structure they transform makes those changes available for inspection and analysis, but reliable adaptation also requires criteria for what must survive.

The project gives this ambition an executable form. Rules can revise rules, selected families continue towards useful results, and histories expose how those results arose. Stability can mean preserving an answer through continuing activity; correctness must still be judged against the task. The contribution is a foundation for studying and building systems whose changing behaviour can be examined with the same care as their data.

# Bibliography


Allender, E. (2001) 'When worlds collide: derandomization, lower bounds, and Kolmogorov complexity', *FST TCS 2001*. Author manuscript.

Amin, N. and Rompf, T. (2018) 'Collapsing towers of interpreters', *Proceedings of the ACM on Programming Languages*, 2(POPL), Article 52. https://doi.org/10.1145/3158140.

Amin, N. (2026) 'A rational defense of reasonable reflection', in *41st Annual Symposium on Logic in Computer Science (LICS 2026)*. LIPIcs 380, pp. 1:1–1:5. https://doi.org/10.4230/LIPIcs.LICS.2026.1.

Bahr, P. (2012) 'Infinitary term graph rewriting is simple, sound and complete', in *23rd International Conference on Rewriting Techniques and Applications*. LIPIcs 15, pp. 69–84. https://doi.org/10.4230/LIPIcs.RTA.2012.69.

Baldan, P. and Raffaetà, A. (2019) 'Minimisation of Event Structures'. arXiv:1907.07042. Primary preprint.

Barendregt, H.P., van Eekelen, M.C.J.D., Glauert, J.R.W., Kennaway, J.R., Plasmeijer, M.J. and Sleep, M.R. (1987) 'Term graph rewriting', in *PARLE: Parallel Architectures and Languages Europe*. LNCS 259, pp. 141–158. https://doi.org/10.1007/3-540-17945-3_8.

Bertsekas, D.P. (1983) 'Distributed asynchronous computation of fixed points', *Mathematical Programming*, 27, pp. 107–120. Primary paper.

Beyhl, T. and Giese, H. (2016) 'Incremental view maintenance for deductive graph databases using generalized discrimination networks', *Electronic Proceedings in Theoretical Computer Science*, 231, pp. 57–71. https://doi.org/10.4204/EPTCS.231.5.

Cirstea, H. and Kirchner, C. (2001) 'The rewriting calculus — Part I', Logic Journal of the IGPL, 9(3), pp. 339-375. doi:10.1093/jigpal/9.3.339.

Claessen, K. and Hughes, J. (2000) 'QuickCheck: a lightweight tool for random testing of Haskell programs', in *Proceedings of ICFP 2000*, pp. 268–279. https://doi.org/10.1145/351240.351266.

Clavel, M. and Meseguer, J. (2002) 'Reflection in conditional rewriting logic', *Theoretical Computer Science*, 285(2), pp. 245–288. https://doi.org/10.1016/S0304-3975(01)00360-7.

Corradini, A., Ghaffari Saadat, M. and Heckel, R. (2020) 'Encoding Incremental NACs in Safe Graph Grammars using Complementation', *Electronic Proceedings in Theoretical Computer Science*, 330, pp. 88–107. Primary paper.

Cousot, P. and Cousot, R. (1977) 'Abstract interpretation: a unified lattice model for static analysis of programs by construction or approximation of fixpoints', in *Proceedings of the 4th ACM SIGACT-SIGPLAN Symposium on Principles of Programming Languages*, pp. 238–252. Author record and paper.

de Moura, L. and Ullrich, S. (2021) 'The Lean 4 theorem prover and programming language', in *Automated Deduction – CADE 28*. LNCS 12699, pp. 625–635. https://doi.org/10.1007/978-3-030-79876-5_37.

Dijkstra, E.W. (1974) ‘Self-stabilizing systems in spite of distributed control’, *Communications of the ACM*, 17(11), pp. 643–644. Author manuscript.

Driscoll, J.R., Sarnak, N., Sleator, D.D. and Tarjan, R.E. (1989) ‘Making data structures persistent’, *Journal of Computer and System Sciences*, 38(1), pp. 86–124. Paper.

Ehrig, H., Prange, U. and Ehrig, K. (2006) *Tutorial on Fundamentals of Algebraic Graph Transformation*. TU Berlin and University of Leicester, 15 July, 73 slides. https://www.cs.le.ac.uk/events/segravis/material/Ehrig-Tutorial4.pdf.

Endrullis, J., Waldmann, J. and Zantema, H. (2006) ‘Matrix Interpretations for Proving Termination of Term Rewriting’, in *Automated Reasoning: IJCAR 2006*. LNCS 4130, pp. 574–588. https://research.vu.nl/ws/portalfiles/portal/2410076/219625.pdf.

Fernández, M., Kirchner, H. and Pinaud, B. (2019) ‘Strategic port graph rewriting: an interactive modelling framework’, *Mathematical Structures in Computer Science*, 29(5), pp. 615–662. https://doi.org/10.1017/S0960129518000270.

Firebase (n.d.) ‘Serve dynamic content and host microservices with Cloud Run’. Official documentation. Accessed 7 October 2026.

Fritsche, L., Lauer, A., Kratz, M., Schürr, A. and Taentzer, G. (2026) ‘Using weakest application conditions to rank graph transformations for graph repair’, *Logical Methods in Computer Science*, 22(1), article 10, pp. 1–51. Primary text.

Füchslin, R.M. and McCaskill, J.S. (2001) ‘Evolutionary self-organization of cell-free genetic coding’, *Proceedings of the National Academy of Sciences*, 98(16), pp. 9185–9190. Primary paper.

Geiselhart, R. and Wirth, F.R. (2016) ‘Relaxed ISS Small-Gain Theorems for Discrete-Time Systems’, *SIAM Journal on Control and Optimization*, 54(2), pp. 423–449. https://doi.org/10.1137/14097286X.

Grädel, E. and Tannen, V. (2017) ‘Semiring Provenance for First-Order Model Checking’. arXiv:1712.01980. Primary preprint.

Grundy, J., Melham, T. and O’Leary, J. (2006) ‘A reflective functional language for hardware design and theorem proving’, *Journal of Functional Programming*, 16(2), pp. 157–196. https://doi.org/10.1017/S0956796805005757.

Gulwani, S. (2011) ‘Automating String Processing in Spreadsheets using Input-Output Examples’, in *POPL*, pp. 317–330. https://doi.org/10.1145/1926385.1926423.

Halpern, J.Y. (2015) ‘A Modification of the Halpern–Pearl Definition of Causality’. arXiv:1505.00162. Primary paper.

Harnad, S. (1990) ‘The Symbol Grounding Problem’, *Physica D*, 42, pp. 335–346. https://doi.org/10.1016/0167-2789(90)90087-6.

Juba, B., Le, H.S. and Stern, R. (2021) ‘Safe Learning of Lifted Action Models’, in *Proceedings of KR 2021*, pp. 379–389. https://doi.org/10.24963/kr.2021/36.

Kawan, C. (2025) *The Small-Gain Condition for Infinite Networks*. arXiv:2503.03925, version 1. Primary text.

Kissinger, A. and Zamdzhiev, V. (2015) ‘Quantomatic: a proof assistant for diagrammatic reasoning’, in Automated Deduction - CADE-25. Cham: Springer (LNCS 9195), pp. 326-336.

Kleene, S.C. (1952) *Introduction to Metamathematics*. Amsterdam: North-Holland.

Kolmogorov, A.N. (1965) ‘Three approaches to the quantitative definition of information’, Problems of Information Transmission, 1(1), pp. 1-7.

Krivine, J.-L. (2007) ‘A call-by-name lambda-calculus machine’, *Higher-Order and Symbolic Computation*, 20(3), pp. 199–207. https://doi.org/10.1007/s10990-007-9018-9.

Lafont, Y. (1997) ‘Interaction combinators’, Information and Computation, 137(1), pp. 69-101. doi:10.1006/inco.1997.2643.

Lamb, C. and Zacchiroli, S. (2021) ‘Reproducible Builds: Increasing the Integrity of Software Supply Chains’, author manuscript, arXiv:2104.06020. https://arxiv.org/abs/2104.06020.

Lamport, L. (1978) ‘Time, clocks, and the ordering of events in a distributed system’, *Communications of the ACM*, 21(7), pp. 558–565. https://doi.org/10.1145/359545.359563.

Lamport, L. (2000) ‘Fairness and hyperfairness’, *Distributed Computing*, 13(4), pp. 239–245. https://doi.org/10.1007/PL00008921.

Lee, C.S., Jones, N.D. and Ben-Amram, A.M. (2001) ‘The size-change principle for program termination’, in Proceedings of the 28th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL ’01). ACM, pp. 81-92. doi:10.1145/360204.360210.

Lee, P. and Necula, G. (1997) ‘Research on Proof-Carrying Code for Mobile-Code Security’, DARPA Workshop on Foundations for Secure Mobile Code, 26–28 March. https://people.eecs.berkeley.edu/~necula/Papers/pcc_fmcs97.pdf.

Lippi, S. (2007) ‘The graphical Krivine machine’, Higher-Order and Symbolic Computation, 20(3), pp. 295–318. doi:10.1007/s10990-007-9011-3. Publisher abstract consulted.

Machado, R., Ribeiro, L. and Heckel, R. (2015) ‘Rule-based transformation of graph rewriting rules: towards higher-order graph grammars’, *Theoretical Computer Science*, 594, pp. 1–23. https://doi.org/10.1016/j.tcs.2015.01.034.

McKay, B.D. and Piperno, A. (2014) ‘Practical graph isomorphism, II’, *Journal of Symbolic Computation*, 60, pp. 94–112. Preprint.

Might, M. and Shivers, O. (2006) ‘Improving Flow Analyses via ΓCFA: Abstract Garbage Collection and Counting’, in *Proceedings of ICFP 2006*, pp. 13–25. Primary paper.

Newman, M.H.A. (1942) ‘On theories with a combinatorial definition of “equivalence”’, *Annals of Mathematics*, 43(2), pp. 223–243. https://doi.org/10.2307/1968867.

Nielsen, M., Plotkin, G. and Winskel, G. (1981) ‘Petri nets, event structures and domains, part I’, *Theoretical Computer Science*, 13(1), pp. 85–108. https://doi.org/10.1016/0304-3975(81)90112-2.

Nogueira, P. and García-Pérez, Á. (2024) ‘Equivalence of eval-readback and eval-apply big-step evaluators by structuring the lambda-calculus’s strategy space’, *Logical Methods in Computer Science*, 20(4), article 24. https://doi.org/10.46298/LMCS-20(4:24)2024.

Peled, D., Vardi, M.Y. and Yannakakis, M. (1999) ‘Black Box Checking’, in *Formal Methods for Protocol Engineering and Distributed Systems*, pp. 225–240. Primary text.

Peters, K. and van Glabbeek, R.J. (2015) 'Analysing and comparing encodability criteria', Electronic Proceedings in Theoretical Computer Science, 190, pp. 46–60. Primary paper.

Plotkin, G.D. (2004) 'A structural approach to operational semantics', *Journal of Logic and Algebraic Programming*, 60–61, pp. 17–139. https://homepages.inf.ed.ac.uk/gdp/publications/sos_jlap.pdf.

Plump, D. (1998) 'Termination of graph rewriting is undecidable', *Fundamenta Informaticae*, 33(2), pp. 201–209. https://doi.org/10.3233/FI-1998-33204.

Plump, D. (2010) 'Checking graph-transformation systems for confluence', *Electronic Communications of the EASST*, 26, 15 pages. https://doi.org/10.14279/tuj.eceasst.26.367.

Plump, D. (2012) 'The design of GP 2', *Electronic Proceedings in Theoretical Computer Science*, 82, pp. 1–16. https://doi.org/10.4204/EPTCS.82.1.

Rensink, A. (2007) 'Isomorphism Checking in GROOVE', *Electronic Communications of the EASST*, 1. https://doi.org/10.14279/tuj.eceasst.1.77.

Reynolds, J.C. (1972) 'Definitional interpreters for higher-order programming languages', in *Proceedings of the ACM Annual Conference*, pp. 717–740. https://doi.org/10.1145/800194.805852.

Riddle, S., Köhler, S. and Ludäscher, B. (2014) 'Towards Constraint Provenance Games', in *6th USENIX Workshop on the Theory and Practice of Provenance*. Primary paper.

Rump, S.M. (2010) 'Verification methods: Rigorous results using floating-point arithmetic', *Acta Numerica*, 19, pp. 287–449. Author manuscript.

Sakar, C. and Kastro, Y. (2018) *Online Shoppers Purchasing Intention Dataset*. UCI Machine Learning Repository. https://doi.org/10.24432/C5F88Q.

Sandmann, C. and Habel, A. (2019) 'Rule-based graph repair', *Electronic Proceedings in Theoretical Computer Science*, 309, pp. 87–104. https://doi.org/10.4204/EPTCS.309.5.

Schewe, K.-D. and Ferrarotti, F. (2022) 'Behavioural theory of reflective algorithms I: Reflective sequential algorithms', *Science of Computer Programming*, 223, Article 102864. https://doi.org/10.1016/j.scico.2022.102864.

Schewe, K.-D. and Ferrarotti, F. (2026) 'Behavioural theory of reflective algorithms II: Reflective parallel algorithms', *Science of Computer Programming*, 253, Article 103505. https://doi.org/10.1016/j.scico.2026.103505.

Shen, A. (2015) 'Around Kolmogorov complexity: basic notions and results', arXiv:1504.04955. https://arxiv.org/abs/1504.04955.

Smith, B.C. (1984) 'Reflection and semantics in Lisp', in *Proceedings of POPL 1984*, pp. 23–35. https://doi.org/10.1145/800017.800513.

Söldner, R. and Plump, D. (2024) 'Formalising the Double-Pushout Approach to Graph Transformation', *Logical Methods in Computer Science*, 20(4), article 3. https://doi.org/10.46298/LMCS-20(4:3)2024.

Stoyle, G., Hicks, M., Bierman, G., Sewell, P. and Neamtiu, I. (2007) 'Mutatis Mutandis: Safe and Predictable Dynamic Software Updating', ACM Transactions on Programming Languages and Systems, 29(4), Article 22. https://mhicks.me/papers/mutatis-journal.pdf.

Tarski, A. (1955) 'A lattice-theoretical fixpoint theorem and its applications', *Pacific Journal of Mathematics*, 5(2), pp. 285–309. https://doi.org/10.2140/pjm.1955.5.285.

Tarski, A. (1956) 'The concept of truth in formalized languages', in *Logic, Semantics, Metamathematics*. Translated by J.H. Woodger. Oxford: Clarendon Press, pp. 152–278.

Tomita, K., Murata, S. and Kurokawa, H. (2007) 'Self-description for construction and computation on graph-rewriting automata', *Artificial Life*, 13(4), pp. 383–396. https://doi.org/10.1162/artl.2007.13.4.383.

Varró, G., Varró, D. and Schürr, A. (2006) 'Incremental graph pattern matching: data structures and initial experiments', *Electronic Communications of the EASST*, 4, pp. 1–15. Paper.

Wills, P.R. (2023) 'Origins of Genetic Coding: Self-Guided Molecular Self-Organisation', *Entropy*, 25(9), article 1281. Primary paper.